

# Container & VM Convergence

## KubeVirt Unified Platform Strategy

The future of infrastructure is running containers and VMs on the same platform. KubeVirt brings VMs into Kubernetes — same kubectl, same CI/CD, same monitoring, same networking. Migrate VMware VMs to KubeVirt with hyper2kvm and run them alongside containers on a single unified platform. One team, one toolchain, one platform.

KubeVirt | Unified Platform | kubectl for VMs | CDI Import | Multus | GPU | Service Mesh

# Why Convergence

Managing two platforms (VMs + containers) doubles complexity, cost, and staffing.

## Two Platforms vs One Platform

| Aspect                | Separate (VMware + K8s)       | Converged (KubeVirt) | Benefit                                 |
|-----------------------|-------------------------------|----------------------|-----------------------------------------|
| Infrastructure        | ESXi cluster + K8s cluster    | Single K8s cluster   | 50% fewer servers, one management plane |
| Networking            | NSX + Calico/Cilium           | Calico/Cilium (both) | Unified network policies for VMs + pods |
| Storage               | vSAN + CSI provider           | CSI provider (both)  | One storage backend, one provisioner    |
| Monitoring            | vROps + Prometheus            | Prometheus (both)    | Unified dashboards, one alerting system |
| CI/CD                 | Separate pipelines            | Same pipeline        | Deploy VMs + containers in one workflow |
| Team skills           | VMware admins + K8s engineers | K8s engineers (all)  | One team, one skill set, cross-trained  |
| Licensing             | vSphere + K8s distro          | K8s distro only      | Eliminate VMware licensing entirely     |
| Annual cost (100 VMs) | \$350K+                       | \$0-\$50K            | \$300K+ savings                         |

### What Runs as Containers

- Microservices (stateless, 12-factor apps)
- APIs and web frontends
- CI/CD runners (Jenkins, GitLab)
- Monitoring stack (Prometheus, Grafana)
- Message queues (Kafka, RabbitMQ)
- New cloud-native applications

### What Runs as VMs (KubeVirt)

- Legacy applications (can't containerize)
- Windows workloads (SQL Server, AD, IIS)
- Databases requiring specific OS tuning
- Appliances (firewalls, load balancers)
- GPU workloads (passthrough)
- Anything needing full OS isolation

# KubeVirt Architecture

VMs as first-class Kubernetes resources — managed with kubectl, scheduled by K8s.

## KubeVirt Component Stack

| Component       | Runs On       | Function                                       | Equivalent                  |
|-----------------|---------------|------------------------------------------------|-----------------------------|
| virt-operator   | Control plane | Manages KubeVirt lifecycle and upgrades        | Like vCenter update manager |
| virt-api        | Control plane | REST API for VM CRUD operations                | Like vCenter API            |
| virt-controller | Control plane | Watches VM objects, ensures desired state      | Like vCenter scheduler      |
| virt-handler    | Every node    | DaemonSet that manages QEMU/KVM on each node   | Like ESXi hostd agent       |
| virt-launcher   | Per VM (pod)  | Wrapper pod that runs QEMU process for each VM | Like VMX process on ESXi    |
| CDI             | Control plane | Containerized Data Importer — imports VM disks | Like vCenter import wizard  |
| virtctl         | CLI (client)  | CLI tool for VM-specific operations            | Like govc or PowerCLI       |

## VM as Kubernetes YAML

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: db-server
  namespace: production
  labels:
    app: database
    tier: backend
spec:
  running: true
  template:
    metadata:
      labels:
        app: database
    spec:
      domain:
        cpu: { cores:4 }
        memory: { guest: 8Gi }
```

```
devices:
  disks:
    - name: rootdisk
      disk: { bus: virtio }
    - name: datadisk
      disk: { bus: virtio }
  interfaces:
    - name: default
      masquerade: {}
resources:
  requests: { memory: 8Gi, cpu: "4" }
networks:
  - name: default
    pod: {}
volumes:
  - name: rootdisk
    persistentVolumeClaim: { claimName: db-server-root }
  - name: datadisk
    persistentVolumeClaim: { claimName: db-server-data }
```

# Migration to KubeVirt

hyper2kvm converts VMware VMs and deploys directly to KubeVirt.

## End-to-End Migration Pipeline

```
# Step 1: Export from VMware
h2kvmctl export --server vcenter.corp.com --vm db-server --output /data/exports/

# Step 2: Convert to QCOW2
h2kvmctl convert --source /data/exports/db-server.vmdk \
  --format qcow2 --fix-all --compress

# Step 3: Upload to KubeVirt via CDI
virtctl image-upload pvc db-server-root \
  --size=50Gi \
  --image-path=/data/exports/db-server.qcow2 \
  --uploadproxy-url=https://cdi-uploadproxy.cdi \
  --insecure

# Step 4: Deploy VM on KubeVirt
kubectl apply -f db-server-vm.yaml

# Step 5: Verify
virtctl console db-server
kubectl get vmi db-server -o wide
# NAME          AGE   PHASE   IP           NODE
# db-server     2m    Running 10.244.1.15  worker-01

# Or: h2kvmctl does steps 2-4 in one command:
h2kvmctl convert --source db-server.vmdk \
  --deploy-k8s --kubevirt-namespace production \
  --kubevirt-storage-class ceph-rbd
```

### VM + Container Communication

- VMs get pod IPs — routable within cluster
- Kubernetes Services expose VMs like pods
- `kubectl expose vmi db-server --port=5432`
- Container apps connect to VM via Service DNS
- `psql -h db-server.production.svc.cluster.local`
- Network policies apply to VMs and pods equally

### Gradual Migration Path

- **Phase 1:** Migrate VMs from VMware → KubeVirt (lift-and-shift)
- **Phase 2:** Containerize stateless apps (extract from VMs)
- **Phase 3:** VMs remain for stateful/legacy only
- **Phase 4:** All new apps are containers, VMs for legacy
- No big-bang containerization needed
- VMs and containers coexist permanently if needed

- Service mesh (Istio) can include VMs

- Migration velocity: your pace, not vendor's deadline

# Unified Operations

One toolchain for containers and VMs — kubectl, Helm, ArgoCD, Prometheus.

## Unified Toolchain

| Tool                 | For Containers                | For VMs (KubeVirt)           | Benefit                                 |
|----------------------|-------------------------------|------------------------------|-----------------------------------------|
| <b>kubectl</b>       | <code>kubectl get pods</code> | <code>kubectl get vmi</code> | Same CLI for everything                 |
| <b>Helm</b>          | Deploy apps via charts        | Deploy VMs via charts        | Version-controlled VM definitions       |
| <b>ArgoCD</b>        | GitOps for containers         | GitOps for VMs               | Git is source of truth for all infra    |
| <b>Prometheus</b>    | Pod metrics                   | VM metrics (virt-exporter)   | Unified monitoring, one Grafana         |
| <b>Velero</b>        | Backup pods + PVCs            | Backup VMs + PVCs            | One backup tool for everything          |
| <b>Cert-Manager</b>  | TLS for services              | TLS for VM services          | Automated certificate management        |
| <b>Istio/Linkerd</b> | Service mesh for pods         | Service mesh includes VMs    | mTLS, traffic management, observability |
| <b>Kyverno/OPA</b>   | Pod policies                  | VM policies                  | Unified policy enforcement              |

### GitOps for VMs — VMs as Code

Store VM definitions in Git alongside container deployments. ArgoCD syncs both. A developer pushes a change to the VM YAML → ArgoCD applies it → KubeVirt updates the VM. Same workflow as container deployments. Infrastructure review via pull requests. Complete audit trail in git history.

# Platform Architecture Patterns

Three proven patterns for converged container + VM platforms.

## Pattern 1: K3s + KubeVirt (Edge/Small)

- **Scale:** 1-3 nodes, 5-30 VMs + containers
- **K8s distro:** K3s (lightweight, single binary)
- **Storage:** Longhorn or local NVMe
- **Network:** Flannel + Multus for VM VLANs
- **Use case:** Edge sites, remote offices, labs
- **Resource overhead:** K3s + KubeVirt = ~2 GB RAM
- **Setup time:** 30 minutes (quickstart.sh)
- **Cost:** \$0 software

## Pattern 2: RKE2/OKD + KubeVirt (Enterprise)

- **Scale:** 5-20 nodes, 50-200 VMs + containers
- **K8s distro:** RKE2 or OKD (OpenShift community)
- **Storage:** Ceph/Rook (replicated, HA)
- **Network:** Calico + Multus + MetalLB
- **Use case:** Production data center
- **HA:** 3 control plane nodes, worker auto-scaling
- **Setup time:** 1-2 days
- **Cost:** \$0 (OKD) or \$50K/yr (OpenShift)

## Pattern 3: OpenShift Virtualization

- **Scale:** 10-100+ nodes, 200+ VMs + containers
- **K8s distro:** OpenShift (Red Hat supported)
- **Storage:** ODF (OpenShift Data Foundation / Ceph)
- **Network:** OVN-Kubernetes + Multus
- **Use case:** Enterprise with support requirements
- **Includes:** Web console, VM templates, migration toolkit
- **Support:** Red Hat 24/7 support included
- **Cost:** \$150K-\$500K/yr (vs \$350K+ VMware)

## Choosing the Right Pattern

- **< 30 VMs, no support needed:** K3s + KubeVirt
- **30-200 VMs, some support:** RKE2 + KubeVirt
- **200+ VMs, full support:** OpenShift Virtualization
- **Air-gapped:** K3s (smallest footprint, offline install)
- **Multi-tenant MSP:** RKE2 with namespace isolation
- **Regulated (SOX/HIPAA):** OpenShift (certified)
- All patterns use hyper2kvm for VM migration
- Start with K3s POC → graduate to enterprise

# Implementation Roadmap

12-month journey from VMware-only to unified container + VM platform.

## Convergence Roadmap

| Quarter | Milestone        | Activities                                                              | Outcome                                                 |
|---------|------------------|-------------------------------------------------------------------------|---------------------------------------------------------|
| Q1      | Foundation       | Deploy K8s cluster, install KubeVirt, POC with 5 VMs, team training     | Working KubeVirt cluster, team comfortable with kubectl |
| Q2      | Migration Wave 1 | Migrate 50 VMs from VMware to KubeVirt, deploy first containerized apps | 50 VMs + 10 container apps on unified platform          |
| Q3      | Migration Wave 2 | Migrate remaining VMs, containerize 10 stateless apps from VMs          | All VMs migrated, VMware decommissioned                 |
| Q4      | Optimization     | GitOps (ArgoCD), service mesh, unified monitoring, DR setup             | Production-grade converged platform, fully automated    |

### One Platform. One Team. Zero VMware License Cost.

KubeVirt eliminates the false choice between containers and VMs. Run both on Kubernetes with unified tooling, monitoring, networking, and storage. hyper2kvm migrates your VMware VMs directly into KubeVirt — same VMs, same data, new platform. Start with a 5-VM POC on K3s in 30 minutes, scale to enterprise with OpenShift when ready. The convergence journey is incremental, reversible, and saves \$300K+ annually.